# Clase 3 - programación

October 10, 2026

[2]:
```
numeros = [3,7,10,15,20]

for numero in numeros:
    if numero >= 10:
        print(numero)
```

```
10
15
20
```

[3]:
```
edad = "25"
if edad >= 18:
    print("Mayor de edad")

#No funciona porque está comparando str con int
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 2
      1 edad = "25"
----> 2 if edad >= 18:
      3     print("Mayor de edad")

TypeError: '>=' not supported between instances of 'str' and 'int'
```

[6]:
```
# Cadenas de texto
nombre = "Python"
mensaje = "Bienvenido a " + nombre + "!"
mensaje
```

[6]:
```
'Bienvenido a Python!'
```

[8]:
```
# Concatenar diferentes tipos de dato
version = 3.12
mensaje = "Estas usando python " + str(version)
mensaje
```

[8]:
```
'Estas usando python 3.12'
```

[9]:
```
# La variable original sigue siendo un float, solo me la convirtió para␣
↪almacenarla en mensaje
version
```

[9]:
```
3.12
```

[10]:
```
# Operaciones con cadenas de texto

texto = "Hola Mundo"
texto
```

[10]:
```
'Hola Mundo'
```

[11]:
```
texto.upper()
```

[11]:
```
'HOLA MUNDO'
```

[12]:
```
texto.lower()
```

[12]:
```
'hola mundo'
```

[17]:
```
texto
```

[17]:
```
'Hola Mundo'
```

[20]:
```
texto.replace("Hola","Adios") # Primer argumento es la cadena que quiero␣
↪reemplazar y segundo es por lo que quiero reemplazar
```

[20]:
```
'Adios Mundo'
```

[22]:
```
nuevo_texto = texto.replace("Hola","Adios")
```

[23]:
```
nuevo_texto
```

[23]:
```
'Adios Mundo'
```

[24]:
```
texto = "Python"
```

[25]:
```
texto[0]
```

[25]:
```
'P'
```

[33]:
```
texto[-1]
```

[33]:
```
'n'
```

[27]:
```
texto[5]
```

[27]: 'n'

[28]:
```
texto[0:3]
```

[28]: 'Pyt'

[29]:
```
texto[1:4]
```

[29]: 'yth'

[30]:
```
texto[:4]
```

[30]: 'Pyth'

[31]:
```
texto[1:]
```

[31]: 'ython'

[38]:
```
texto[0]+texto[4]
```

[38]: 'Po'

[39]:
```
texto[0:5:4]
```

[39]: 'Po'

[40]:
```
frase = ["Hola", "como", "estas"]
```

[41]:
```
frase[0] + " " + frase[2] + " " + frase[1]
```

[41]: 'Hola estas como'

[42]:
```
# LISTAS
```

[43]:
```
mi_lista = [1,2,3,"Python", True]
mi_lista[0]
```

[43]: 1

[44]:
```
mi_lista[-1]
```

[44]: True

[45]:
```
mi_lista.append(5) # para agregar elementos al final de la lista
mi_lista
```

[45]: [1, 2, 3, 'Python', True, 5]

[46]:
```
mi_lista.insert(3,4) # para insertar un valor en una posición específica
mi_lista
```

[46]:
```
[1, 2, 3, 4, 'Python', True, 5]
```

[47]:
```
mi_lista.append(2)
mi_lista
```

[47]:
```
[1, 2, 3, 4, 'Python', True, 5, 2]
```

[48]:
```
mi_lista.remove(2)# Quita el primer elemento que haga match con el valor que le␣
↪di
mi_lista
```

[48]:
```
[1, 3, 4, 'Python', True, 5, 2]
```

[49]:
```
mi_lista.pop() # sin argumento regresa y quita el ultimo elemento
```

[49]:
```
2
```

[50]:
```
mi_lista
```

[50]:
```
[1, 3, 4, 'Python', True, 5]
```

[51]:
```
mi_lista.pop(0) # con argumento en este caso 0 quita el elemento 0
```

[51]:
```
1
```

[52]:
```
mi_lista
```

[52]:
```
[3, 4, 'Python', True, 5]
```

[53]:
```
mi_lista.sort()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[53], line 1
----> 1 mi_lista.sort()

TypeError: '<' not supported between instances of 'str' and 'int'
```

[56]:
```
numlist = [7,3,5,1,2] # solo las listas de elementos iguales pueden ordenarse␣
↪con sort() y reverse()
```

[57]:
```
numlist
```

[57]:
```
[7, 3, 5, 1, 2]
```

[59]:
```
numlist.sort() # me ordena de menor a mayor
numlist
```

[59]:
```
[1, 2, 3, 5, 7]
```

[60]:
```
numlist.reverse() # me ordena de mayor a menor
numlist
```

[60]:
```
[7, 5, 3, 2, 1]
```

[61]:
```
strlist = ["c","o","a","d"] #tambien funciona en cadenas de texto
strlist
```

[61]:
```
['c', 'o', 'a', 'd']
```

[63]:
```
strlist.sort()
strlist
```

[63]:
```
['a', 'c', 'd', 'o']
```

[64]:
```
strlist.reverse()
strlist
```

[64]:
```
['o', 'd', 'c', 'a']
```

[66]:
```
lista = [1, "dos", False, [45, "cien"]]
```

[67]:
```
lista[3][0]
```

[67]:
```
45
```

[68]:
```
# TUPLAS
```

[69]:
```
mi_tupla = (1,2,3, "Python")
```

[71]:
```
mi_tupla[0]
```

[71]:
```
1
```

[72]:
```
mi_tupla.count(2) # cuenta cuántas veces existe ese valor dentro de la tupla
```

[72]:
```
1
```

[73]:
```
mi_tupla.index("Python") # me regresa el indice del elemento que contiene ese␣
↪valor
```

[73]:
```
3
```

[74]:
```
mi_tupla
```

[74]:
```
(1, 2, 3, 'Python')
```

[75]:
```
list(mi_tupla) # las tuplas se pueden convertir en listas
```

[75]:
```
[1, 2, 3, 'Python']
```

[76]:
```
tuple(strlist) # las listas se pueden convertir en tuplas
```

[76]:
```
('o', 'd', 'c', 'a')
```

[77]:
```
# DICCIONARIO
```

[78]:
```
mi_dict = {"nombre":"Python", "version":3.12, "activo":True} # para crear␣
 ↪diccionario de usan llaves
mi_dict["nombre"]
```

[78]:
```
'Python'
```

[79]:
```
mi_dict["version"] # acceso a los elemetos por su clave
```

[79]:
```
3.12
```

[80]:
```
mi_dict["año"] = 2026 # si una clave no existe y yo se la doy con asignación me␣
 ↪agrega la clave y el valor
mi_dict
```

[80]:
```
{'nombre': 'Python', 'version': 3.12, 'activo': True, 'año': 2026}
```

[81]:
```
mi_dict["version"] = 3.14 #modificando un valor que ya existia
mi_dict
```

[81]:
```
{'nombre': 'Python', 'version': 3.14, 'activo': True, 'año': 2026}
```

[82]:
```
mi_dict.keys()  # me da todas las claves
```

[82]:
```
dict_keys(['nombre', 'version', 'activo', 'año'])
```

[83]:
```
for i in mi_dict.keys(): # puedo iterar con las claves
    print(i)
```

```
nombre
version
activo
año
```

[84]:
```
mi_dict.values() # me da los valores
```

[84]:
```
dict_values(['Python', 3.14, True, 2026])
```

[85]:
```
for i in mi_dict.values(): # puedo iterar con los valores
    print(i)
```

```
Python
3.14
True
2026
```

[86]:
```
mi_dict.items()
```

[86]:
```
dict_items([('nombre', 'Python'), ('version', 3.14), ('activo', True), ('año', 2026)])
```

[88]:
```
for clave, valor in mi_dict.items(): # items me da pares de clave valor, por␣
 ↪eso puedo iterar con 2 variables
    print(clave + " : " + str(valor))
```

```
nombre : Python
version : 3.14
activo : True
año : 2026
```

[89]:
```
mi_dict.pop("nombre") # pop elimina el elemento que coincida con la clave que␣
 ↪le di
```

[89]:
```
'Python'
```

[90]:
```
mi_dict
```

[90]:
```
{'version': 3.14, 'activo': True, 'año': 2026}
```

[91]:
```
mi_dict.clear() # deja el diccionario vacio
```

[92]:
```
mi_dict
```

[92]:
```
{}
```

[93]:
```
materias = {}
materias["lunes"] = [6103, 7540]
materias["martes"] = [6201]
materias["miércoles"] = [6103, 7540]
materias["jueves"] = []
materias["viernes"] = [6201]
```

[94]:
```
materias["lunes"][1]
```

[94]:
```
7540
```

[95]:
```
materias["domingo"] # si quiero ingresar a una clave que no existe, me da error
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[95], line 1
----> 1 materias["domingo"]

KeyError: 'domingo'
```

[98]:
```
materias.get("domingo") # para obtener un valor con su clave pero que no me de␣
↪error si no existe.
```

[99]:
```
# ---- Ejercicios practica
```

[100]:
```
calificaciones = [85,90,75,100]
```

[101]:
```
calificaciones.append(95)
```

[102]:
```
len(calificaciones) # Cuántos elementos tiene una colección ya sea lista o tupla
```

[102]: 5

[103]:
```
calificaciones[0]
```

[103]: 85

[104]:
```
coordenadas = (20.67, -103.35)
```

[105]:
```
estudiante = {"nombre":"Ana", "edad":30, "calificacion":95}
```

[106]:
```
estudiante["nombre"]
```

[106]: 'Ana'

[107]:
```
estudiante["calificacion"]
```

[107]: 95

[108]:
```
memo = {"nombre":"Guillermo", "edad":36, "comida":"Morisqueta", "Python":True}
```

[109]:
```
print(memo["comida"])
```

```
Morisqueta
```

[113]:
```
ventas = [100,200,300,400]
total = 0
for venta in ventas:
    total = total + venta
print(total)
```

```
1000
```

[111]:
```
venta
```

[111]:
```
400
```

[114]:
```
calificaciones = [50,80,95,60,75]
aprobados = []
for calificacion in calificaciones:
    if calificacion >= 70:
        aprobados.append(calificacion)
print(aprobados)
```

```
[80, 95, 75]
```

[115]:
```
calificaciones = [50,80,95,60,75]
aprobados = []
for calificacion in calificaciones:
    if calificacion < 70:
        aprobados.append(calificacion)
print(aprobados)
```

```
[50, 60]
```

[116]:
```
# Introducción a funciones
```

[117]:
```
print("Hola Ana")
print("Hola Pedro")
print("Hola Luis")
```

```
Hola Ana
Hola Pedro
Hola Luis
```

[121]:
```
def saludar(nombre):
    print("Hola " + nombre)
```

[120]:
```
saludar("Ana")
saludar("Pedro")
saludar("Luis")
```

```
Hola Ana
Hola Pedro
Hola Luis
```

[122]:
```
def calcular_area(base, altura):
    area = base * altura
    print(area)
```

[123]:
```
calcular_area(5,10)
```

```
50
```

[124]:
```
calcular_area(7,8)
```

```
56
```

[125]:
```
# PRINT vs RETURN
```

[126]:
```
def sumar(a,b):
    print(a + b)
```

[127]:
```
resultado = sumar(5,3)
print(resultado)
```

```
8
None
```

[128]:
```
def sumar(a,b):
    return a + b
```

[129]:
```
resultado = sumar(5,3)
print(resultado)
```

```
8
```

[130]:
```
resultado * 2
```

[130]:
```
16
```

[131]:
```
# print() muestra información.
# return devuelve información para que pueda utilizarse después.
```

[138]:
```
# --- clasificador de calificaciones
def clasificar_calificacion(calificacion):
    if calificacion == 100:
        return "Perfecto"
    elif calificacion >= 90:
        return "Excelente"
    elif calificacion >= 70:
        return "Aprobado"
    else:
        return "Reprobado"
```

[139]:
```
clasificar_calificacion(100)
```

[139]:
```
'Perfecto'
```

[134]:
```
clasificar_calificacion(75)
```

[134]:
```
'Aprobado'
```

[135]:
```
clasificar_calificacion(50)
```

[135]:
```
'Reprobado'
```

[141]:
```
# -- Filtrar calificaciones
def obtener_aprobados(calificaciones):
    aprobados = []
    for calificacion in calificaciones:
        if calificacion >= 70:
            aprobados.append(calificacion)
    return aprobados
```

[142]:
```
calificaciones = [50,80,95,60,75]
obtener_aprobados(calificaciones)
```

[142]:
```
[80, 95, 75]
```

[143]:
```
obtener_aprobados([100,86,72,60,98])
```

[143]:
```
[100, 86, 72, 98]
```

[144]:
```
# -- Filtrar calificaciones reprobatorias
def obtener_reprobados(calificaciones):
    aprobados = []
    for calificacion in calificaciones:
        if calificacion < 70:
            aprobados.append(calificacion)
    return aprobados
```

[145]:
```
obtener_reprobados(calificaciones)
```

[145]:
```
[50, 60]
```

[148]:
```
# -- Filtrar calificaciones con cal aprobatoria
def obtener_aprobados(calificaciones, aprob):
    aprobados = []
    for calificacion in calificaciones:
        if calificacion >= aprob:
            aprobados.append(calificacion)
    return aprobados
```

[149]:
```
obtener_aprobados(calificaciones, 80)
```

[149]:
```
[80, 95]
```

[150]:
```
obtener_aprobados(calificaciones, 50)
```

[150]:
```
[50, 80, 95, 60, 75]
```

[151]:
```
# -- Total de ventas
# Instrucciones:
# Crear una función que reciba una lista de ventas y devuelva la suma total.

def calcular_total(ventas):
    total = 0
    for venta in ventas:
        total = total + venta
    return total
```

[153]:
```
calcular_total([100,200,300])
```

[153]:
```
600
```

[154]:
```
# -- Promedio de ventas
# Instrucciones:
# Crear una función que reciba una lista de ventas y devuelva el promedio.

def promedio(ventas):
    suma = calcular_total(ventas)
    resultado = suma / len(ventas)
    return resultado
```

[155]:
```
promedio([100,200,300])
```

[155]:
```
200.0
```

[158]:
```
# -- Revisar código

def obtener_mayores(numeros):
    mayores = []
    for numero in numeros:
        if numero > 10:
            mayores.append(numero)
    return mayores
```

[159]:
```
obtener_mayores([5,10,15,20,25])
```

[159]:
```
[15, 20, 25]
```

[160]:
```
def obtener_medios(numeros):
    medios = []
    for numero in numeros:
        if numero < 15 and numero > 7:
            medios.append(numero)
    return medios
```

[161]:
```
obtener_medios([5,8,11,14,20,25])
```

[161]:
```
[8, 11, 14]
```

[162]:
```
def ejemplo_or(numeros):
    resultado = []
    for numero in numeros:
        if numero > 20 or numero < 12:
            resultado.append(numero)
    return resultado
```

[163]:
```
ejemplo_or([5,8,11,14,20,25])
```

[163]:
```
[5, 8, 11, 25]
```

[172]:
```
def agenda(nombres, telefonos):
    if len(nombres) != len(telefonos):
        print("Las listas son de diferentes tamaños")
    else:
        dicc = {}
        for indice in range(len(nombres)):
            dicc[nombres[indice]]=telefonos[indice]
        return dicc
```

[174]:
```
agenda(["Memo","Irving","Luis"], ["3319876543","123891738372","12830982"])
```

[174]:
```
{'Memo': '3319876543', 'Irving': '123891738372', 'Luis': '12830982'}
```

[ ]:
```

```